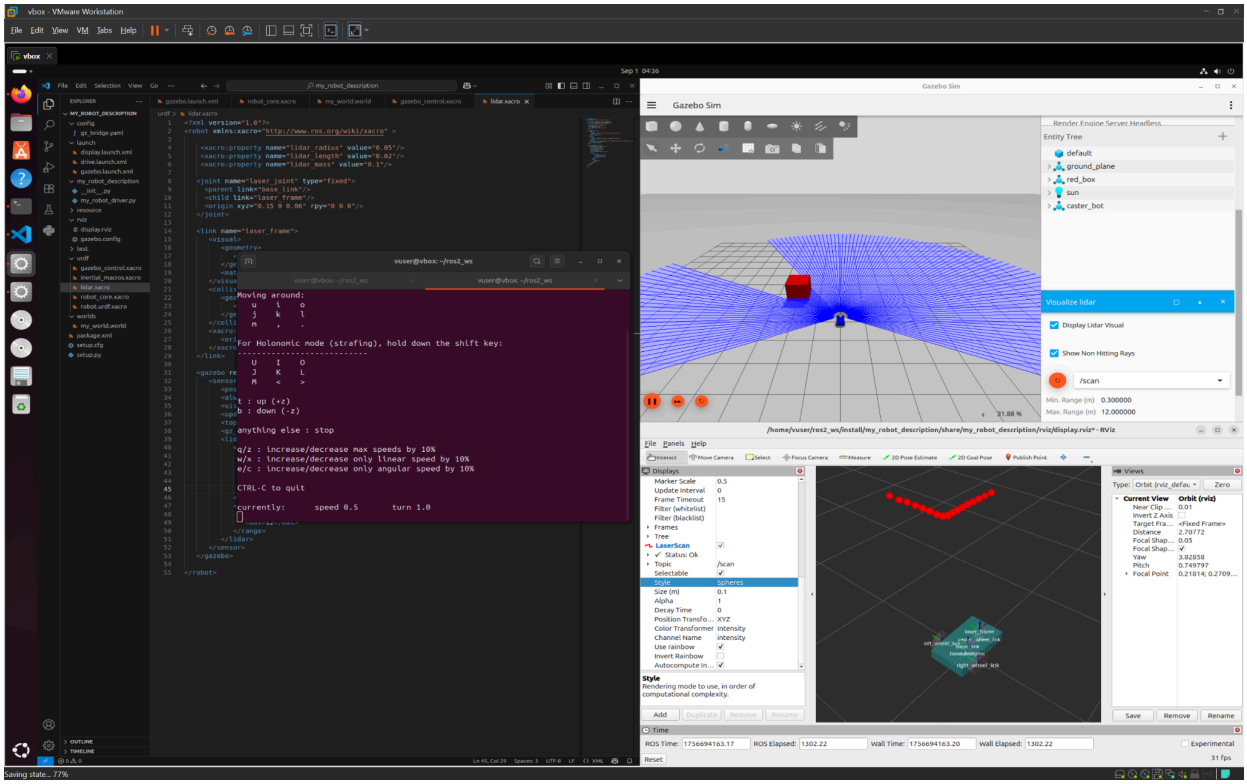
Assignment 2

This assignment builds upon your work from Assignment 1. You will transition from a static robot model to a dynamic simulation in Gazebo. The primary goals are to enhance your URDF using Xacro for modularity, integrate your robot with the Gazebo simulator, implement teleoperation control, and add a sensor to perceive the environment. This will provide you with foundational skills in robot simulation and sensor integration within the ROS 2 ecosystem.



Task Breakdown

Task 1: Modularizing with Xacro

Your current URDF file provides a detailed description of the robot. However, you likely noticed significant repetition, especially in the wheel definitions. Xacro (XML Macros) is a tool that allows you to create more readable, maintainable, and reusable URDF files.

[Using Xacro to clean up your code — ROS 2 Documentation: Jazzy documentation]

- Convert URDF to Xacro:
    - Rename your .urdf file to .urdf.xacro.

    - Your first task is to identify repetitive patterns in the description of your robot. Dimensions are a prime example. See if you can't standardize a few.

```
<xacro:property name="base_radius"      value="0.16"/>
<xacro:property name="base_height"      value="0.06"/>
```

      Example

    - Using the Xacro documentation, create a macro for a wheel link and joint. This macro should take parameters for properties like name (e.g., left, right) and position.
    - Use this macro to generate the left and right wheels for your robot. This probably won't make your robot description any shorter, as this is quite a simple robot. However, for most robots with a certain level of complexity, we can save a significant amount of time and code lines.
- Document Your Reasoning:
    - In your report, explain the sections of your URDF that you chose to modularize with Xacro macros and how you did so.
    - Justify your choices. Why did these specific parts benefit from being turned into macros?
- Verification in RViz:
    - Update your launch file to use the new .urdf.xacro file. You will need xacro to be part of the command that loads the robot description.
    - Launch RViz and visualize your robot. It should look identical to the robot from Assignment 1.
- REMEMBER TO TAKE SNAPSHOTS

Task 2: Spawning in Gazebo and Teleoperation

Now you will bring your robot to life in the Gazebo simulator.

SDF worlds — Gazebo documentation

Building your own robot — Gazebo documentation

Moving the robot — Gazebo documentation

Use ROS 2 to interact with Gazebo — Gazebo documentation

Sensors — Gazebo documentation

Launch Gazebo from ROS 2 — Gazebo documentation

- Create a Custom World:
    - Create a new world file (.world) for Gazebo and place it in a “worlds” folder. This world should be simple: include a ground plane and a few basic shapes (e.g., a box, a cylinder, a sphere) scattered around the origin. This will give you objects to detect later.
    - Refer to the Gazebo documentation for creating simple world files.
    - Note that you’ll need some gazebo plugins in your world file for the physics and sensor system to function correctly. Have a look at the “Preliminaries” in the sensor documentation and the sensor-specific plugins.
- Gazebo Integration:
    - You will need to add physics-related tags to your Xacro file, particularly for <collision> and <inertial> properties if they are not already specified.
    - Additionally, the friction of the caster wheel should be set close to zero so it doesn’t prohibit the robot from moving. You can (and often should) also set a friction coefficient for the wheels as well, though the default is usually ok unless we’re emulating an actual physical robot.
    - You will also need to add a Gazebo plugin for the differential drive controller (gz-sim-diff-drive-system). Remember that all gazebo-specific plugins used in urdf files need to have <gazebo> tags around them.
        - Since we’re removing the node that publishes joint states, the left and right wheels won't have any published transforms. However, gazebo publishes joint states internally. We can use the gz-sim-joint-state-publisher-system plugin to publish these through to ROS itself.
    - You will need to configure the ROS-Gazebo bridge (ros_gz_bridge) to pass a few internal Gazebo topics through to ROS and some from ROS through to Gazebo. The best way to do this is to have a config file in a config folder that the bridge node reads in your launch file.
        - cmd_vel: To control the robot's movement (from ros to gz).

- clock: To synchronize ROS 2 nodes with the simulation time in gz.
- joint_states: To visualize the robot's moving parts correctly in RViz2.
- odom: To get the robot's estimated position and velocity..
- tf: To define the spatial relationship between all coordinate frames.
- scan: To get the LIDAR sensor data.

```
! gz_bridge.yaml
# gz topic subscribed to by DiffDrive plugin
- ros_topic_name: "cmd_vel"
  gz_topic_name:  "cmd_vel"
  ros_type_name:  "geometry_msgs/msg/Twist"
  gz_type_name:   "gz.msgs.Twist"
  direction:       ROS_TO_GZ

# gz topic published by Sensors plugin
- ros_topic_name: "scan"
  gz_topic_name:  "scan"
  ros_type_name:  "sensor_msgs/msg/LaserScan"
  gz_type_name:   "gz.msgs.LaserScan"
  direction:       GZ_TO_ROS
```

Correct indentation is very important in config files. Note the “-”

```
<!-- ROS <-> Gazebo bridge -->
<node pkg="ros_gz_bridge" exec="parameter_bridge">
  <param name="config_file" value="$(var bridge_config_path)"/>
</node>
```

Example

- Update your launch file to start Gazebo with your custom world and spawn your robot.

```
<arg name="headless" default="false" description="Run Gazebo without the GUI client"/>

<executable cmd="gz sim -s -r $(var world_path)" name="gz_server" output="screen"/>

<executable cmd="gz sim -g" name="gz_client" output="screen" unless="$(var headless)"/>
```

Example (gz_server computes the physics, gz_client is the GUI, and can be neglected if visualizing in rviz)

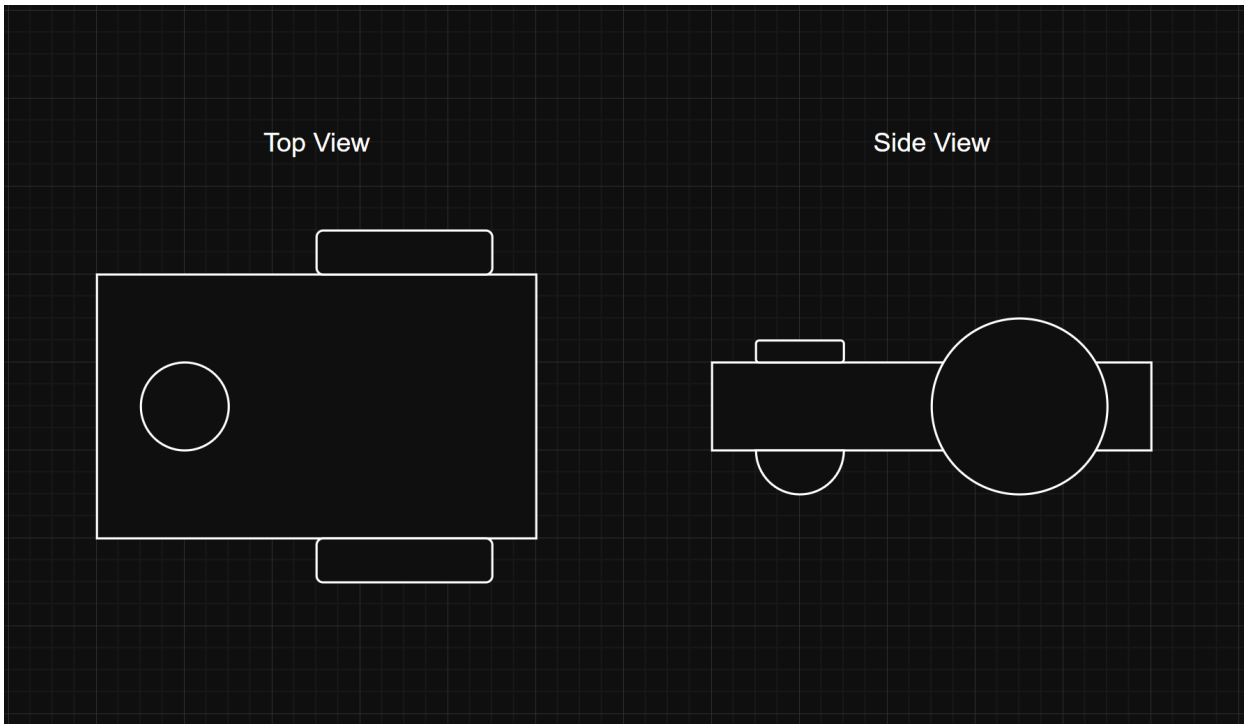
- Keyboard Control:

- The goal is to drive your robot around the Gazebo world using your keyboard.
- Install the teleop_twist_keyboard package: sudo apt-get install ros-jazzy-teleop-twist-keyboard.
    - In Gazebo, at the top right, there are three dots that, if clicked, you can search teleop and use an internal Gazebo controller.
- Run the teleop_twist_keyboard node in a separate terminal. You should now be able to drive your robot around the simulated world if the diff drive controller and cmd_vel bridging are correctly configured.

Task 3: Adding a LiDAR Sensor

To enable your robot to "see" its environment, you will add a simulated LiDAR sensor.

[Sensors — Gazebo documentation]



- Add LiDAR Link:
    - In your .urdf.xacro file, create a new link for the LiDAR sensor. A simple cylinder is a good visual representation.
    - Attach this link to the base_link with a fixed joint, positioning it on top of the robot

chassis.

- Configure the Gazebo Sensor:
    - Add a <sensor> tag associated with the LiDAR link (within a <gazebo> tag).
    - Configure it to use the gpu_lidar sensor plugin as described in the Gazebo sensor documentation.
    - Ensure the sensor is configured to publish its data on a ROS 2 topic (e.g., /scan).

```
<gazebo reference="laser_link">
  <sensor name="laser" type="gpu_lidar">
    <pose> 0 0 0 0 0 0 </pose>
    <always_on>true</always_on>
    <visualize>true</visualize>
    <update_rate>30</update_rate>
    <topic>scan</topic>
    <gz_frame_id>laser_link</gz_frame_id>
    <lidar>
      <scan>
        <horizontal>
          <samples>45</samples>
          <min_angle>-${pi/2}</min_angle>
          <max_angle>${pi/2}</max_angle>
        </horizontal>
      </scan>
      <range>
        <min>0.1</min>
        <max>10</max>
      </range>
    </lidar>
  </sensor>
</gazebo>
```

Example

    - In Gazebo, just like with the internal teleop GUI, you can also add a lidar visualizer if you click the top right three dots and search for lidar. Then, in the GUI, scroll down and refresh topics; if the lidar is correctly configured /scan should be selectable.

- A Note on Virtual Machines:
    - Gazebo can be graphically intensive. If you experience rendering issues or crashes within your VM, you may need to disable hardware rendering. You can do this by running the following command in your terminal before launching Gazebo:
      export LIBGL_ALWAYS_SOFTWARE=1 (or adding it to .bashrc once)

Task 4: Visualizing Sensor Data

The final step is to visualize the data from your new LiDAR sensor in RViz.

- Configure RViz:
    - Add a LaserScan display and subscribe it to the topic where your LiDAR data is being published.
    - You should see the points from the LiDAR scan corresponding to the shapes in your Gazebo world.
- Dynamic Visualization:
    - Drive your robot around using the teleop keyboard node and observe how the point cloud in RViz changes as the robot moves closer to or further away from the objects in your world.
    - Capture a screenshot that shows both Gazebo and RViz side-by-side. In Gazebo, the robot should be facing an object, and in RViz, the corresponding laser scan from the LiDAR should be clearly visible.

Delivery

- Your Xacro, with an explanation of which parts you modularised and why
- Snapshots verifying the Xacro robot still looks right in RViz
- A screenshot of the robot driving in your Gazebo world
- A side-by-side Gazebo and RViz screenshot showing the LiDAR scan matching the world
- Key snippets: your bridge config, the diff-drive plugin block, and the LiDAR sensor block
- What every report must contain

I cannot stress this enough: a report that is a wall of text with no code and no pictures does not show me that you did the work or that you understood it. A few Assignment 1 submissions were essentially a piece of paper: no screenshots, no code, no evidence. That will not be approved. Here is what I am actually looking for.

- Screenshots of your progress. At minimum, a screenshot at each major step, and the specific ones listed above. I want to see your robot, your terminal, your RViz and Gazebo windows: proof that it ran on your machine.
- Code snippets of the important bits. Show me the parts that matter, your launch file, the key YAML and Xacro. Snippets are welcome and expected, but keep them to the parts that carry the idea, not the entire codebase.
- A short reflection. What did you try, what worked, what did not, and why? An honest analysis of a failure matters to me as much as a clean success.
- Structure and length. A short intro, your design choices and method, results with the evidence above, and the reflection. Aim for roughly 1,000 to 2,000 words; I am not looking for 15,000. I want a well-structured report that conveys the necessary information in a professional, elegant way.

Channel your favorite science communicator. I should finish your report convinced you understand what you built, not just that you compiled something. Don't worry if you haven't solved every edge case; I am looking for effort, understanding, and a willingness to learn. We have plenty more to cover before the semester is over.

Formatting. I strongly recommend writing your report in LaTeX (Overleaf is great for this and works well in groups). Here is some layout inspiration, including how to add code snippets cleanly: https://www.overleaf.com/read/gstrwxnkwnhy#0160ba

On using AI and LLMs. LLMs are allowed as a co-pilot for research and debugging, the same way you would use Google, the docs, or a knowledgeable friend. What they are not for is doing all your work and writing your report for you. The whole point of these assignments is that you build an understanding of how these systems fit together; if you outsource that, you have outsourced the learning. So use AI to get unstuck, then make sure you can explain every line you submit. Please briefly declare where and how you used AI in your report; being upfront about it is part of good engineering practice.

"Approved" means: a genuine attempt at all the tasks, plus a report that meets the requirements above and shows me your thinking. Good luck! :)